Design and Implementation of a Real-Time AI Sign Language Translator Using Hand-Landmark Tracking and Deep Learning

Sarnala Sasidhar, Adabala Likitha Naidu, Mamidi Jaya Prakash, and Mudavath Charan Tej
Department of Artificial Intelligence and Data Sciences
Parul University, Vadodara, Gujarat, India
Project guide: Ms. Binisha Makwana

***Abstract*—Deaf and hard-of-hearing people often cannot find an interpreter when they need one, for instance at a clinic, a government office, or in an emergency. This paper reports how we built a small web application that reads hand gestures from an ordinary camera and shows them as text, with optional spoken output. A webcam stream is passed through MediaPipe Hands, which returns 21 landmarks per hand. These landmarks are normalized and sent to a TensorFlow/Keras classifier, and the predictions are filtered by a confidence threshold and a short voting window before the text is displayed. The back end is a Flask-style REST service packaged with Docker. We explain the requirements, the architecture, the feature and decision rules, and the privacy issues that come with processing camera input, and we compare the design with three recent landmark-based systems. We have not reported accuracy or latency for our prototype, since our records do not include a documented dataset, a signer-wise split, or a repeatable benchmark. Instead we set out the metrics and test protocol that a proper measurement needs, and we list what is still missing.**

***Index Terms*—sign language recognition, MediaPipe, hand landmarks, TensorFlow, assistive technology, computer vision, human–computer interaction, Docker**

I. INTRODUCTION

Sign languages have their own vocabulary, grammar, and regional variants, and many deaf and hard-of-hearing people rely on them every day [1, 2]. Most hearing people never learn any sign language, so a conversation across that gap often depends on an interpreter. Interpreters are expensive and usually have to be booked ahead, which is no help at a hospital reception desk, a bank counter, or a late-night emergency. Typing on a phone or passing handwritten notes works, but it is slow and interrupts the flow of talking.

Hand tracking has become light enough to run on a laptop. Rather than feeding raw pixels to a network, a program can find the hand first and keep only a few dozen numbers describing its pose, so most of the clutter from the background, clothing, and skin tone never reaches the classifier [3, 4]. A small network built with a standard framework [5] can then turn those numbers into a sign label, and a container image [6] lets another person run the same application without repeating the setup.

This paper describes the AI Sign Language Translator, our final-year project, which follows that recipe. It watches the user's hands through a webcam, extracts landmarks, classifies the pose, smooths the result over a few frames, and prints the text on screen. If the user wants, it also reads the text aloud. The first version covers the manual alphabet and a few common words such as "hello," "thank you," and "help."

A. Problem Statement

We want a low-cost program that recognizes hand gestures from a live camera and turns them into text or speech in near real time. It should need no gloves, depth sensors, or interpreter, and it should be easy to start on a machine it has never run on before.

B. Contributions

This paper offers the following:

1. a requirements list and scope for a camera-only sign-to-text-and-speech tool, with the things we decided not to build;
2. an architecture that keeps offline training apart from live inference, with the browser client, REST service, landmark extractor, classifier, and speech output as separate parts;
3. exact definitions of the feature vector, the confidence threshold, and the smoothing rule used for live recognition;
4. a discussion of deployment, security, and privacy for a tool that handles camera data; and
5. a verification plan, with metrics and protocols for a later measured study, and a plain statement of which results we do *not* have yet.

C. Organization

Section II covers related work and Section III the requirements. The architecture is in Section IV and the way we ran the project in Section V. Sections VI and VII give the recognition design and the implementation. Section VIII explains how the system should be verified, Section IX deals with security and privacy, Section X lists limitations and future work, and Section XI concludes.

II. BACKGROUND AND RELATED WORK

A. Sign Language Recognition

One of the first vision-based systems recognized American Sign Language with hidden Markov models, using a camera mounted on a desk and later one worn on a cap [7]. Later surveys split the problem into hand and body tracking, feature extraction, and classification or translation. They also point out that the hands carry only part of the message, since facial expression, head movement, and posture add grammatical information [2, 8].

For our purposes the key split is between *isolated* recognition and *continuous* translation. In the isolated case a single pose or

short clip gets one label, which is an ordinary classification task. Continuous translation has to cut a video stream into signs, cope with the way neighbouring signs blend together, and then write a grammatical sentence in a spoken language. Camgoz et al. treat this as a sequence-to-sequence problem [9]. It is a much harder task than recognizing letters or words, and the available data are small compared with speech corpora.

B. Landmark-Based Hand Tracking

MediaPipe Hands runs a palm detector first and then a second model that predicts 21 three-dimensional keypoints for every hand it finds [3]. It sits on top of the general MediaPipe framework for perception pipelines [4]. The output is a short list of numbers, so whatever comes after it can be small and quick, and it is less affected by the background or the lighting. The price is that any tracking mistake passes straight to the classifier. When fingers overlap, the hand moves fast, or part of the hand is hidden, the landmarks become unreliable.

C. Sequence Models and Transformers

Signs that involve movement call for models that read a sequence of landmark vectors. Gated recurrent networks such as the LSTM [10] cope with sequences of different lengths, and attention-based models [11] capture long-range links; most current translation systems are built on the latter. WLASL, a large word-level American Sign Language dataset, is a common benchmark for isolated signs [12]. When a system outputs sentences, its quality is usually scored with n-gram overlap measures such as BLEU [13].

D. Recent Landmark-Based Systems

Table I lists three recent systems that pair MediaPipe-style landmarks with a learned classifier. The numbers are the ones the authors themselves give, and we repeat them only for context [14–16]. Maia et al. pass keypoint sequences to a Transformer that predicts gloss tokens, and a fine-tuned BART model then writes the sentence. They report competitive results on the PHOENIX14T weather corpus but clearly weaker ones on How2Sign, where no glosses exist, and the model is too heavy to run in real time on ordinary hardware [14]. Utomo et al. train a recurrent network on landmark sequences for 26 letters and 10 digits of Indonesian sign language. They report about 97% accuracy on a small custom set, though the vocabulary is limited and the data are not public [15]. Bansod et al. cut out the hand with MediaPipe and classify the crop with YOLOv5 [17]. They report about 96% accuracy on 30 classes at roughly 32 ms per frame, but this speed needs a GPU [16].

E. Gap Addressed

The systems above either reach high accuracy on small vocabularies collected privately, or tackle full translation at a cost that ordinary laptops cannot pay. Our project sits somewhere in between. It is an engineering prototype: a complete camera-to-speech pipeline made from open components, easy to extend, and described together with an account of what we have and have not measured.

III. SCOPE AND REQUIREMENTS

A. Scope

The translator is a web page that talks to a Python inference service. It takes live video from a webcam or phone camera, finds the hands, and recognizes isolated signs from a small starting vocabulary. The text appears in a panel beside the video, and a text-to-speech engine can read it out. Buttons start and stop the camera and clear the output. We expect it to be used indoors, under normal lighting, on an ordinary laptop.

We left several things out of this version: translation of whole sentences with sign-language grammar, non-manual markers such as facial expression, support for more than one sign language, and a native mobile app. A text-to-sign direction, where typed text is shown as signs by an animated avatar, is sketched in the design but not built.

B. Users

The main users are deaf and hard-of-hearing people who want to be understood by people who do not sign. Others who might use it are members of the public, teachers and students in mixed classrooms, people learning sign language, and staff at clinics and service counters.

C. Functional Requirements

Table II gives the functional (FR) and non-functional (NFR) requirements that shaped the design. The numbers in the NFRs were targets set at the start. They are not measured results.

IV. SYSTEM ARCHITECTURE

A. Overview

The application is split into a browser client and a single backend service. The client shows the video and the results. The service receives a frame, or a landmark vector computed elsewhere, runs hand detection and classification, and replies in JSON. Training happens offline and produces a model file that the service loads when it starts. Fig. 1 shows how the pieces fit.

B. Offline Training Pipeline and Online Inference Pipeline

We keep training and inference apart for two reasons. The running service stays small because it holds only the trained model and none of the training code, and the training data can be handled under tighter rules than a live camera feed. Fig. 2 shows both pipelines. In training, labeled images or short clips become landmark vectors, go through the same normalization used at run time, are split into sets, and are used to train and evaluate the classifier, which is then exported. In live use the service repeats the landmark and normalization steps on each frame, applies the confidence threshold and the smoothing window, and maps the class index to text.

C. Request Flow

Fig. 3 shows one recognition cycle. The client grabs a frame from the video element, encodes it, and posts it to `/predict`. The service decodes the image, converts it from BGR to RGB, and looks for a hand. If it finds none, it returns an error, which the page shows as a plain “no hand detected” message, and the loop carries on. If it finds one, the landmarks of the first hand are

TABLE I

COMPARISON OF RECENT LANDMARK-ASSISTED SIGN RECOGNITION SYSTEMS (VALUES AS STATED BY THE ORIGINAL AUTHORS)

Work	Input / features	Model	Vocabulary	Reported result	Main limitation
Maia et al. [14]	Body, hand, and face keypoints from MediaPipe	Transformer sign-to-gloss, then BART gloss-to-text	Sentence-level (PHOENIX14T, How2Sign)	Competitive BLEU-4 on PHOENIX14T; lower without gloss labels	Needs gloss labels; heavy compute; narrow domain
Utomo et al. [15]	21 hand landmarks per frame, sequences	Stacked LSTM classifier	36 classes (A–Z, 0–9)	≈97% accuracy, macro-F1 ≈96%	Small vocabulary; private dataset
Bansod et al. [16]	MediaPipe hand region, cropped image	YOLOv5-small with transfer learning	30 classes (letters and four word signs)	≈96% accuracy; ≈32 ms/frame on GPU	GPU-dependent; one-handed, clear signs only
This work	21 landmarks × 3 coordinates (63 values)	Compact Keras classifier plus temporal smoothing	Alphabet and common words (initial)	No accuracy claimed (Sec. VIII)	Isolated signs; signer-dependent data

TABLE II

REQUIREMENTS AND DESIGN RESPONSE

ID	Requirement	Design response
FR1	Capture live video from a camera	Browser camera API; OpenCV in tools
FR2	Detect and track hand landmarks	MediaPipe Hands
FR3	Classify gestures with a trained model	Keras model on 63-value vectors
FR4	Show recognized text in real time	Output panel updated per response
FR5	Optional speech output	Text-to-speech toggle
FR6	Start, stop, and clear controls	Large UI buttons
FR7	Report unrecognized gestures and keep running	"No hand"/"unknown" state, loop continues
NFR1	Near-real-time response, about 15–30 frames/s	Light model; frame skipping
NFR2	Accuracy above 85% under normal lighting (target)	Evaluation protocol, Sec. VIII
NFR3	Usable without training	Minimal two-panel UI
NFR4	Run on standard laptops (Windows, Linux)	Container image; CPU inference
NFR5	Add gestures by retraining	Separate training pipeline
NFR6	No video stored without consent	Stateless inference; no frame logging

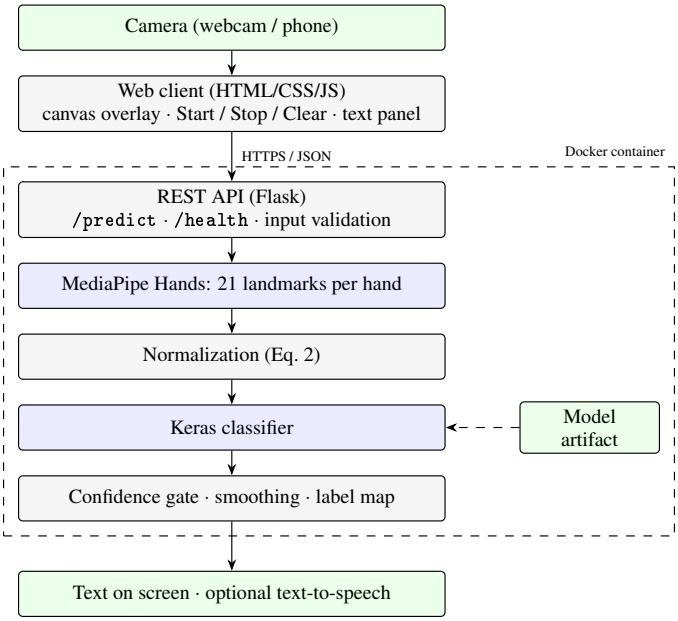


Fig. 1. Logical architecture of the translator. The dashed boundary marks the containerized backend.

flattened into a 63-value vector, normalized, and classified, and the post-processing step sends back the label with its confidence.

D. Reverse Direction (Design Only)

The design leaves room for a text-to-sign direction. Typed or spoken input would become text, be simplified by a light language-processing step, be matched word by word against a sign dictionary, and be shown as recorded clips or by an animated avatar. We describe it only to show how the modules could be extended. It is not part of the prototype.

E. Technology Selection

Table III lists the main components and why we picked them. Python has the best machine-learning libraries, OpenCV [18] handles the images, MediaPipe supplies the landmarks, and TensorFlow with Keras trains the classifier. Flask [19] keeps the API small. FastAPI would be a reasonable alternative because it validates requests automatically and supports asynchronous handling. Docker makes the deployment repeatable.

V. DEVELOPMENT METHODOLOGY

A. Iterative Process

We worked in short Agile-style iterations [20], which fit a project where experiments keep changing the plan. Each iteration produced one module, such as landmark extraction, data collection, classifier training, the interface, or packaging, and ended with a check against the requirements in Table II. Problems at the joins between modules therefore showed up early.

B. User-Centric Principles

Since deaf users are the main audience, we made the interface visual and plain: a video panel next to a text panel, large labelled buttons, readable fonts, and enough colour contrast to follow accessibility guidance [21]. The system tracks only the hands and not the whole body, which reduces the computation and leaves the signer a small area to keep in frame. The result appears straight away, so a signer can tell at once whether a sign was read and try again if it was not.

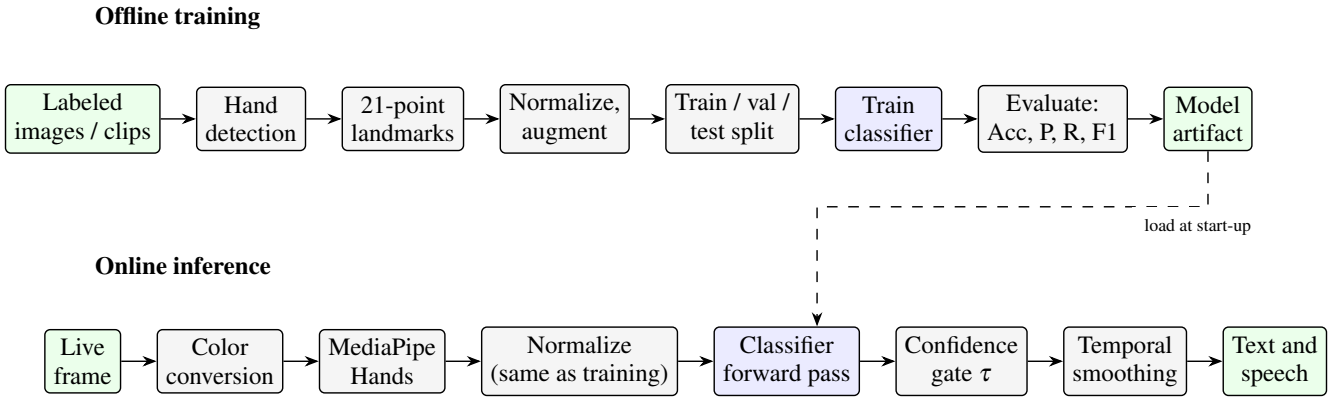


Fig. 2. Offline training pipeline (top) and online inference pipeline (bottom). The same normalization is applied in both, which avoids train–serve skew.

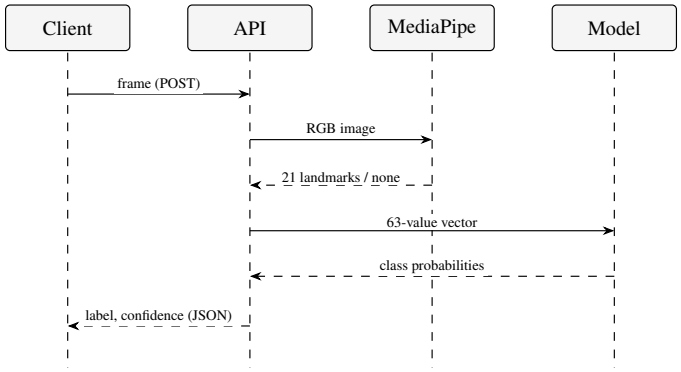


Fig. 3. Message sequence for one recognition request.

C. Quality Assurance Strategy

The plan has several levels. Unit tests check small things, for example that landmark flattening always gives 63 numbers. Integration tests send a known image through the whole pipeline. System tests run under realistic conditions, acceptance sessions involve real users, and regression checks run whenever the model changes. Section VIII says which of these are documented and which are still to be done.

D. Project Timeline

The schedule ran for twelve weeks (Fig. 4). Requirements took week 1 and design week 2. Data collection filled weeks 3 and 4, then model training ran from week 5 to 7 alongside interface work in weeks 6 and 7. We integrated in week 8, tested in weeks 9 and 10, built the container in week 11, and wrote the documentation in week 12.

VI. Recognition Design

A. Feature Representation

For every hand it finds, MediaPipe returns 21 landmarks $\mathbf{p}_i = (x_i, y_i, z_i)$ for $i = 0, \ldots, 20$, where index 0 is the wrist. The x and y values are scaled by the image width and height, and z is a relative depth. Joining them gives the raw feature vector

$$\mathbf{f} = [x_0, y_0, z_0,\ x_1, y_1, z_1,\ \ldots,\ x_{20}, y_{20}, z_{20}]^\top \in \mathbb{R}^{63}. \qquad (1)$$

TABLE III
Technology Stack and Rationale

Layer	Technology	Rationale
Language	Python 3	Rich ML and vision libraries
Video	OpenCV	Capture, color conversion, decoding
Hand tracking	MediaPipe Hands	Ready 21-point landmarks, real-time
Model	TensorFlow / Keras	Training and inference
Numerics	NumPy, scikit-learn	Arrays, splits, metrics
API	Flask (FastAPI alt.)	Lightweight REST service
Client	HTML, CSS, JavaScript	Camera access, UI
Speech	pyttsx3 / gTTS	Text-to-speech
Packaging	Docker	Portable, repeatable runtime
Collaboration	Git, GitHub, Trello	Versioning, tasks

Two hands would give 126 values, but the prototype uses only the first hand it detects, so two-handed signs are a known weak point.

B. Normalization

If raw image coordinates went straight into the classifier, the same hand shape would look different at different places in the frame or at different distances from the camera. So we measure every landmark from the wrist and divide by the largest wrist-to-landmark distance:

$$\tilde{\mathbf{p}}_i = \frac{\mathbf{p}_i - \mathbf{p}_0}{s}, \qquad s = \max_j \|\mathbf{p}_j - \mathbf{p}_0\|_2. \qquad (2)$$

This removes the effect of position and size. It does not remove rotation or mirroring, so for left-handed signers we rely on data augmentation. The same function has to be used in training and in the service, or accuracy measured offline will not hold up in use.

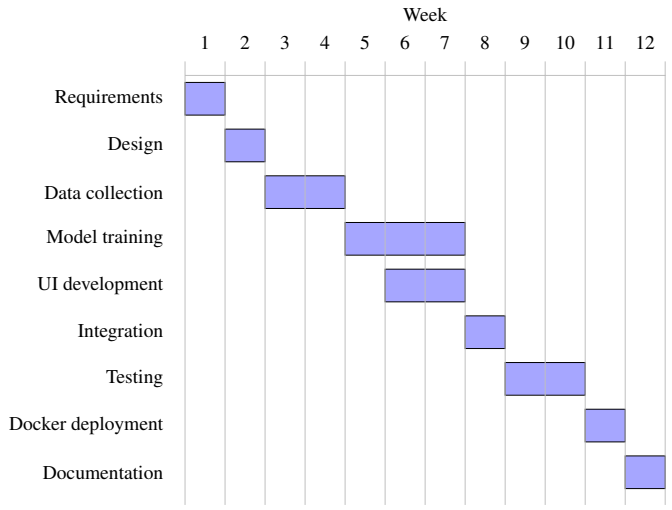


Fig. 4. Twelve-week project schedule.

C. Classifier

For static signs, a small feed-forward network turns $\tilde{\mathbf{f}}$ into C class scores $\mathbf{z}$, and a softmax converts those scores into probabilities:

$$P(c \mid \tilde{\mathbf{f}}) = \frac{\exp(z_c)}{\sum_{k=1}^{C} \exp(z_k)}. \tag{3}$$

Training minimizes categorical cross-entropy. We do not fix the number or width of the layers here, because they are hyper-parameters to be chosen on a validation set. Signs with movement could use the landmarks of T consecutive frames fed to a recurrent [10] or attention-based [11] model. That belongs to the roadmap, since the starting alphabet and word set does not need it.

D. Confidence-Gated Decision

A closed-set classifier always picks some label, even when the hand is doing something random. To allow an "unknown" answer we compare the top probability with a threshold τ:

$$\hat{y} = \begin{cases} \arg\max_c P(c \mid \tilde{\mathbf{f}}), & \max_c P(c \mid \tilde{\mathbf{f}}) \geq \tau, \\ \text{unknown}, & \text{otherwise.} \end{cases} \tag{4}$$

A low τ lets wrong signs through and a high one rejects correct signs, so it should be tuned on validation data. Softmax confidence is also not a calibrated probability, which we leave for future work.

E. Temporal Smoothing

Frame-by-frame predictions flicker while the hand moves from one pose to another. We therefore keep the last W gated predictions and output a label only if it occurs at least m times in that window:

$$\text{emit}(c) \iff \sum_{t=n-W+1}^{n} \mathbb{1}[\hat{y}_t = c] \geq m, \quad m > W/2. \tag{5}$$

A held letter should be added only once, so the text composer also waits for the label to change, or for a short pause, before it accepts the same letter again. A larger W makes the output steadier but delays it by about W frame intervals.

F. Text Composition and Speech

Accepted labels go into a text buffer. Letters build up words, a word sign is inserted whole, and the clear button empties the buffer. With speech switched on, the current word or phrase goes to an offline text-to-speech engine, so no text has to be sent to an outside service.

G. Evaluation Metrics

Take C classes, and let TP_c, FP_c, and FN_c be the true positives, false positives, and false negatives for class c, with N test samples in total. The verification protocol uses

$$\text{Acc} = \frac{1}{N} \sum_{c=1}^{C} TP_c, \tag{6}$$

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{7}$$

$$F1_c = \frac{2 P_c R_c}{P_c + R_c}, \quad \text{F1}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^{C} F1_c. \tag{8}$$

The delay for one frame is the sum of its stages,

$$T = T_{\text{cap}} + T_{\text{net}} + T_{\text{mp}} + T_{\text{clf}} + T_{\text{post}} + T_{\text{ui}}, \tag{9}$$

If the stages run as a pipeline the frame rate is limited by the slowest one, $1/\max(T_{\text{stage}})$, and if they run one after another it is $1/T$. A sentence-level version would also need word error rate, $\text{WER} = (S + D + I)/N_w$, and BLEU [13].

VII. Implementation

A. Backend

The back end is a Python web service. It loads the MediaPipe Hands object and the Keras model once, when it starts, so that no request pays the start-up cost. MediaPipe runs in video mode with up to two hands and detection and tracking confidence of 0.5. The `/predict` route takes an uploaded image, decodes it with OpenCV, converts BGR to RGB, runs detection, and returns a JSON object holding the predicted class index. If no hand is found, it returns a client-error status and a short message. A `/health` route lets a load balancer or the container runtime check that the service is up. The core of the prediction path looks like this, shortened:

```
res = hands.process(cv2.cvtColor(frame,
                    cv2.COLOR_BGR2RGB))
if not res.multi_hand_landmarks:
    return jsonify(error="No hand detected"), 400
hand = res.multi_hand_landmarks[0]
feats = [v for lm in hand.landmark
           for v in (lm.x, lm.y, lm.z)]   # 63 values
x = np.array(feats).reshape(1, -1)
label = int(np.argmax(model.predict(x)))
return jsonify(prediction=label)
```

In the full design, the normalization of Eq. (2), the threshold of Eq. (4), and the smoothing of Eq. (5) sit between feature extraction and the response.

B. Frontend

The client is a static HTML and JavaScript page. It asks for camera permission, draws the live stream, and can overlay the detected hand skeleton on a canvas. Requests go to the server at a limited rate so that it is not flooded. A text panel shows the text so far and the latest label with its confidence. Three buttons start the camera, stop it, and clear the text, and a toggle switches speech on. Video and translation sit side by side, so the signer can check the result without looking far away.

C. Containerization and Deployment

The service is packaged on a slim official Python image. The Dockerfile installs the dependencies listed in a requirements file, copies in the code and the model, exposes the port, and starts the web server on all interfaces. The same image can run on a laptop, a cloud virtual machine, or a managed container platform. A reverse proxy can handle TLS and pass requests on to the container, which matters because browsers allow camera access only on secure origins, apart from localhost. For continuous integration, a repository workflow can build the image, run the unit tests, and push the image to a registry.

D. Error Handling and Robustness

The page tells four situations apart and reports each differently: no camera, no hand in view, a hand whose confidence is below τ, and a server error. The capture loop keeps going in all four, so a short failure does not force the user to restart. The upload route should also check file type and size before it decodes anything.

VIII. Verification Approach and Observations

A. Method

Our verification is based on the design and the code, not on a benchmark. We compared the documented design with the requirements, wrote down the tests and metrics that a controlled evaluation would use, and recorded what evidence exists. Our project notes include some early remarks about performance, but they do not give the dataset size, the class balance, who the signers were, how the data were split, the hardware, or the workload, and without those nobody could reproduce a figure. Quoting numbers here would suggest a precision we cannot back up, so we leave them out.

B. Planned Test Suite

Table IV lists example test cases. Positive cases use clear signs, and negative cases check that random hand movement is rejected. Edge cases cover occlusion, poor lighting, and other borderline input, and interface cases cover the buttons. The tests should run in an isolated environment, check the status code, the response format, and the output text, and be repeatable against a recorded model version.

C. Recommended Evaluation Protocol

A fair measurement of recognition quality should follow these steps. (1) Use a dataset whose classes, sample counts, and signers are documented. (2) Split by *signer* and not by random frame. Neighbouring frames from one person look almost the same, so a random split inflates accuracy, and the test set should come from people who never appeared in training. (3) Report accuracy, per-class precision and recall, macro-F1, and a confusion matrix using Eqs. (6)–(8), with confidence intervals over repeated runs. (4) Pick τ, W, and m on a validation set, then fix them. (5) Measure latency stage by stage with Eq. (9), and report a percentile such as the 95th over a stated number of requests, together with the hardware, software versions, and cache state. (6) For concurrency, increase the number of virtual users with a load-testing tool and record errors and resource use. (7) For usability, recruit deaf and hearing participants through an approved process and record task completion, errors, time on task, and comments.

TABLE IV
Representative Test Cases

Type	Scenario	Input	Expected
+	Clear single sign	Frame showing a vocabulary sign	Correct label
+	Word sign	Frames of “hello” held steadily	“hello” appended once
−	Out-of-vocabulary motion	Random hand movement	“unknown”, nothing appended
−	Empty scene	Frame with no hand	“No hand” message; loop continues
Edge	Partial occlusion	Hand partly out of frame	No crash; low confidence or unknown
Edge	Low light	Dim frame	Graceful degradation
Edge	Two hands	Both hands visible	Defined behavior (first hand)
UI	Controls	Start, stop, clear	State changes as specified
Sec.	Bad upload	Oversized or non-image file	Rejected with error code

D. Status of Evidence

Table V matches each objective with the status our evidence supports. “Designed” means the architecture and procedure are written down, and “implemented” means working code exists in the project. Where the table says a quantity was not measured, no documented experiment backs a number.

E. Observations

Three points came out of reviewing the design. First, a landmark front end is smaller and friendlier to privacy than raw images, and it keeps the classifier light enough that CPU-only use looks realistic. Second, the quality of the whole pipeline is limited by the hand tracker. When fingers hide each other the features get worse, whatever classifier follows. Third, our architecture uploads camera frames to the server. That is fine when the server runs on the user’s own machine, but it conflicts with the requirement of local processing when the server is hosted elsewhere. Extracting landmarks in the browser and sending only the 63 normalized numbers would remove the frame upload and cut both bandwidth and privacy exposure. We have put this on the roadmap.

TABLE V
OBJECTIVES AND EVIDENCE STATUS

Objective	Status	Evidence boundary
Camera-to-text pipeline	Implemented	Code path and flow documented
Hand tracking	Implemented	Uses MediaPipe landmarks
Gesture classification	Implemented	Accuracy not measured here
Speech output	Implemented	Local engine; quality not rated
Real-time operation	Designed	Latency not measured here
Basic UI	Implemented	No formal usability study
Containerized deployment	Designed	Image recipe specified
Sentence translation	Not met	Out of current scope
Multiple languages	Not met	Future work
Accessibility audit	Not done	Planned

TABLE VI
THREAT-ORIENTED CONTROL CHECKLIST

Asset	Threat	Control
Camera frames	Interception in transit; unwanted retention	HTTPS only; do not store or log frames; prefer client-side landmarks
Consent	Camera used without clear permission	Explicit browser prompt; visible camera indicator; notice
Accounts (if added)	Credential disclosure	Salted adaptive hashing (e.g., bcrypt); secure sessions
API	Oversized or malformed uploads; open CORS	Type and size limits; restrictive origin list
Container	Excess privilege; stale packages	Non-root user; slim image; dependency updates
Model	Biased errors across signers	Diverse data; per-group error reporting

F. Validity Considerations

Landmark-based accuracy depends a great deal on who is signing, on the camera angle, and on the lighting, so results from a few signers say little about people in general. Accuracy on isolated letters also tells us little about conversation, where grammar, facial expression, and speed all matter. Last, the figures from other studies in Table I come from different data and cannot be used to rank our system.

IX. SECURITY, PRIVACY, AND RESPONSIBLE USE

A program that watches someone's hands also sees their face and the room behind them, so camera data should be handled as sensitive. Table VI lists the main risks and the controls to put in place and check before any public release. It comes from reading through the design. We did no penetration testing, and the controls are requirements, not proof that protection already exists.

Technical controls are not enough on their own. The tool should present itself as an aid and not as a substitute for a qualified interpreter, especially in medical or legal situations where a wrong translation can do harm. It should admit uncertainty by showing an unknown state or the confidence, instead of always printing a confident word. Training data should include signers of different skin tones, ages, hand sizes, and handedness, and should be collected with informed consent. Deaf people should also take part in design reviews and testing.

X. LIMITATIONS AND FUTURE WORK

A. Limitations

The prototype has several clear limits. It recognizes isolated signs from a small vocabulary and does not handle continuous signing or sign grammar. It ignores facial expression and body movement. Only the first detected hand reaches the classifier, so two-handed signs work poorly. Our training data are small and come from few signers, so we do not know how it behaves with new signers, backgrounds, or lighting. No accuracy, latency, concurrency, or usability results are established by our records. Sign languages differ from country to country, so the vocabulary is tied to one language until the model is retrained. Softmax confidence is uncalibrated. Finally, the reference design uploads frames to the server, which conflicts with a strict local-processing goal when the server is remote.

B. Roadmap

The first job is to run the protocol of Section VIII on a documented dataset split by signer, and to publish the results with the exact model and code version. Next we want landmark extraction to run in the browser, so that only numbers leave the device, and we want sequence models for moving signs and two-hand input. After that comes continuous recognition with sentence output, evaluated on corpora such as PHOENIX14T with WER and BLEU. Smaller items include quick calibration for a new signer, calibrated confidence scores, more sign languages, an avatar for the text-to-sign direction, captions on video calls or in augmented reality, adjustable caption size, and a phone version. Each should come with its own tests and metrics.

XI. CONCLUSION

We built an AI Sign Language Translator that turns hand gestures from a camera into text and optional speech. It uses MediaPipe hand landmarks, a TensorFlow/Keras classifier, a confidence threshold, a short voting window, a REST service, and a Docker image. Training is kept separate from live inference, and the compact landmark input keeps the computation modest. Having a working pipeline is not the same as showing that it works well for real users. We can claim the pipeline. Showing that it works needs a test set split by signer, latency and load measurements, and a usability study with deaf participants, which we have specified in this paper but not yet carried out. Once those are done, the system can serve as a practical base for low-cost tools that help people who sign communicate with people who do not.

ACKNOWLEDGMENT

We thank our project guide, Ms. Binisha Makwana, the faculty of the Department of Artificial Intelligence and Data Sciences at Parul University, and our classmates for their guidance and support during the project.

REFERENCES

[1] World Federation of the Deaf, "Sign language and deaf culture." [Online]. Available: https://wfdeaf.org/

[2] R. Rastgoo, K. Kiani, and S. Escalera, "Sign language recognition: A deep survey," *Expert Syst. Appl.*, vol. 164, Art. no. 113794, 2021.

[3] F. Zhang *et al.*, "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.

[4] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[5] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Syst. Design Implement.*, 2016, pp. 265–283.

[6] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, 2014.

[7] T. Starner, J. Weaver, and A. Pentland, "Real-time American Sign Language recognition using desk and wearable computer-based video," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 12, pp. 1371–1375, Dec. 1998.

[8] S. C. W. Ong and S. Ranganath, "Automatic sign language analysis: A survey and the future beyond lexical meaning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 6, pp. 873–891, Jun. 2005.

[9] N. C. Camgoz, S. Hadfield, O. Koller, H. Ney, and R. Bowden, "Neural sign language translation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7784–7793.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] A. Vaswani *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.

[12] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison," in *Proc. IEEE Winter Conf. Appl. Comput. Vis.*, 2020, pp. 1459–1469.

[13] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318.

[14] W. F. Maia, A. M. Lopes, and S. A. David, "Automatic sign language to text translation using MediaPipe and transformer architectures," *Neurocomputing*, 2025.

[15] W. Utomo *et al.*, "Indonesian language sign detection using MediaPipe with LSTM algorithm," *J. Inform. Web Eng.*, 2025.

[16] D. Bansod *et al.*, "Real-time deep learning-based sign language translator to text using YOLOv5 and MediaPipe," *Int. J. Adv. Res. Comput. Commun. Eng.*, 2025.

[17] G. Jocher *et al.*, "YOLOv5 by Ultralytics," software repository, 2020. [Online]. Available: https://github.com/ultralytics/yolov5

[18] G. Bradski, "The OpenCV library," *Dr. Dobb's J. Softw. Tools*, 2000.

[19] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/

[20] K. Beck *et al.*, "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org/

[21] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/